

Final Demonstration Checklist — Student Management System

This document confirms that the Student Management System meets all completion criteria defined in the project SOP, and explains how each requirement was verified.

✓ Application starts without errors

Backend:

```
bash
cd backend
venv\Scripts\activate
python manage.py runserver
```

Server starts cleanly at (`http://localhost:8000`) with no errors in the console.

Frontend:

```
bash
cd frontend
npm run dev
```

App starts cleanly at (`http://localhost:5173`) and loads without console errors.

✓ Database connection works correctly

- SQLite database (`db.sqlite3`) is created automatically via Django migrations:

```
bash
python manage.py migrate
```

- Confirmed connectivity by successfully running CRUD operations that persist across server restarts.
 - The dashboard's "SQLite DB — Ready & Seeded" indicator confirms an active connection.
-

✓ Create operation works

- Verified via the **Enroll New Student** form in the UI: submitting valid data creates a new record, which immediately appears in the student table and increments the "Total Students" count.
- Verified independently via the API:

```
POST /api/students/
```

with a valid JSON body, returning `201 Created` and the new student object.

✓ Read/list operation works

- The student table displays all existing records with ID, name, contact, course, date of birth, and status.
- Verified via the API:

```
GET /api/students/      → returns full list  
GET /api/students/{id}/ → returns a single record
```

✓ Update operation works

- Verified via the UI: clicking the edit (pencil) icon on a student, changing a field (e.g., status), and saving updates the record and reflects the change immediately in the table and dashboard counts.
- Verified via the API:

```
PATCH /api/students/{id}/
```

returning `200 OK` with the updated object.

✓ Delete operation works

- Verified via the UI: clicking the delete (trash) icon removes the student from the table and decrements the "Total Students" count.
- Verified via the API:

```
DELETE /api/students/{id}/
```

returning `204 No Content`.

- Tested deleting an invalid/non-existent ID, which correctly returns `404 Not Found`.
-

✓ Validation works

Client-side:

- Required fields (name, email, phone, date of birth, course) cannot be submitted empty.
- Email field enforces valid email format before submission.

Server-side (independent of the frontend):

- Duplicate email addresses are rejected with a clear error message, enforced at both the serializer and database (unique constraint) level.
 - Missing required fields are rejected with `400 Bad Request` and field-specific error messages.
 - Verified directly via Postman by sending invalid/duplicate requests that bypass the React form entirely, confirming validation is not solely client-side.
-

✓ Search/filter works

- Search bar filters the student list in real time by name, email, or course.
 - Course dropdown filter narrows the list to a selected course.
 - "Clear" button resets all filters.
-

✓ API endpoints can be demonstrated

All endpoints were tested and can be demonstrated live via Postman:

Operation	Method	Endpoint
Register	POST	<code>/api/auth/register/</code>
Login	POST	<code>/api/auth/login/</code>
Logout	POST	<code>/api/auth/logout/</code>
Current user	GET	<code>/api/auth/me/</code>
List students	GET	<code>/api/students/</code>
Get one student	GET	<code>/api/students/{id}/</code>
Create student	POST	<code>/api/students/</code>
Update student	PUT/PATCH	<code>/api/students/{id}/</code>
Delete student	DELETE	<code>/api/students/{id}/</code>

Full request/response examples are documented in [API_DOCUMENTATION.md](#).

Authentication was also verified: requests to `/api/students/` without a valid token correctly return `401 Unauthorized`, confirming the API — not just the UI — enforces access control.

✅ Student can explain the architecture and code flow

Architecture:

```
User → React Frontend (Vite) → REST API (Django REST Framework)
      → Token Authentication → ORM (Django Models) → SQLite Database
```

Code flow for a typical request (e.g., creating a student):

1. User fills out the **Enroll New Student** form in React.
2. On submit, the frontend runs client-side validation, then sends a `POST` request via Axios to `/api/students/`, with the auth token attached in the `Authorization` header.
3. Django REST Framework's `StudentViewSet` receives the request and passes the data to `StudentSerializer` for validation (required fields, unique email).
4. If valid, the serializer saves a new `Student` model instance via Django's ORM, which writes to the SQLite database.
5. The API returns the created record as JSON with a `201 Created` status.

6. The frontend receives the response, closes/resets the form, and re-fetches the student list so the UI reflects the new record immediately.

The same request/response/ORM pattern applies to Read, Update, and Delete, differing only in HTTP method and the serializer/queryset logic used.

✓ Source code and documentation are submitted

- Full source code (backend + frontend) pushed to GitHub: `[insert repo link]`
 - `README.md` — project overview, setup instructions, architecture, data model
 - `API_DOCUMENTATION.md` — full endpoint reference with request/response examples
 - This file (`DEMONSTRATION_CHECKLIST.md`) — completion verification
 - Screenshots of the UI, forms, and validation errors included in `/docs/screenshots`
-

Completion Statement

All four CRUD operations have been implemented and demonstrated, frontend-backend-database communication is confirmed reliable, validation and testing are complete on both client and server sides, authentication protects all student data, and the source code and full documentation have been submitted to the project repository.